Tampere University

COMP.CS.530-2025-2026-2 Capstone project on LLM fine-tuning

Case 21: Mind Map -Automated Educational Concept Mapping

Jaakko Rajala

Samith Binda Pantho

23 March, 2026

Table of Contents

1. Project Overview 3
   - Summary of Objectives 3
   - Problem & Solution 3
     - Existing Problem 3
     - Possible Solution 3
2. System Overview 3
   - Technical Stack 3
   - Architecture Overview 4
3. Technical Implementation 4
   - Key Features 4
   - Core Logic and Workflow 5
4. Advanced AI Capability: Model Fine-Tuning 6
   - Initial Challenges with Base Models 6
   - Dataset Creation 6
   - Fine-Tuning Process 6
5. Development Journey: Experimentation & Iteration 7
   - Evidence of Experimentation 7
   - Key Challenges & Solutions 7
6. Comparison between Llama3.2 and Kiran2.0 9
   - Summary Statistics 9
   - Performance Analysis 9
7. Lessons Learned & Future Work 11
   - Key Takeaways 11
   - Future Work 11
8. Conclusion 11

1. Project Overview

Summary of Objectives

AgenticContentFlow is an advanced, full-stack educational platform designed to automate the evaluation of student-created concept flow graphs (mindmaps). The primary objective is to provide students with immediate, high-quality feedback on their understanding of course materials by comparing their visual mental models against core concepts extracted directly from educational documents. The system aims to bridge the gap between static content and active, visual knowledge representation.

Problem & Solution

Existing Problem

Traditional assessment of mindmap is a manual, time-consuming process for educators. It is often subjective, lacks scalability, and provides slow feedback loops. For students, this means long waiting times for corrective guidance, which can plateau their learning.

Possible Solution

Mindmap addresses this by providing an AI-driven, automated validation layer. By leveraging Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG), the system:

- Extracts main topics from course documents.
- Uses semantic similarity to match student topics.
- Employs a hybrid scoring engine to evaluate both the presence of topics and the quality of connections.

This ensures consistency, objectivity, and real-time pedagogical feedback.

2. System Overview

Technical Stack

Layer	Technology	Reason
Frontend	React 18, TypeScript, Vite, ReactFlow, Zustand, ELK Layout	ReactFlow provides a robust canvas API; ELK ensures consistent hierarchical visualization.

Backend	Python 3.10, FastAPI, SQLAlchemy, Pydantic	FastAPI offers high performance and excellent integration with AI/ML libraries.
Database	PostgreSQL 16 + pgvector	pgvector allows for efficient semantic search within the relational database.
Embeddings	all-MiniLM-L6-v2	A balance of speed and semantic accuracy for local execution.
LLM Inference	Ollama	For running small models locally and that ensures data privacy and zero cost per token.
Fine-Tuning	Unsloth, Google Colab	Optimized training and cost-effective GPU usage for model specialization.

Architecture Overview

The system follows a modern decoupled architecture:

1. **Frontend:** A responsive React application featuring a drag-and-drop mindmap editor powered by ReactFlow. It uses ELK for automatic layout and Zustand for state management.

2. Backend: A FastAPI service that manages document uploads, RAG processing, and flow evaluation.

3. Data Layer: PostgreSQL with the pgvector extension stores both traditional relational data (topics, evaluations) and high-dimensional document embeddings.

4. AI Layer: Ollama runs locally to handle sensitive educational content, performing both topic extraction and final flow validation.

3. Technical Implementation

Key Features

- **Intelligent RAG Pipeline**: Uploaded documents are chunked into 500-character segments with 10% overlap, embedded, and stored.
- **AI Topic Extraction**: On document upload, the LLM extracts core concepts and their supporting details using a specialized prompt and regex-based post-processing to ensure structured DB storage.

- **Dynamic Flow Validation**: Student graphs are compared semantically. The system uses a Cosine Similarity threshold (default 0.7) to determine if a student's topic matches an expected one.
- **Hybrid Scoring Algorithm**: The system uses a 50/50 blend of:
    - Rule-based Score: Deterministically penalizes missing core topics (up to 70% of the score) and missing details (up to 30%).
    - LLM Score: The LLM analyzes structural quality, connection logic (directionality and hierarchy), and conceptual depth.

Core Logic and Workflow

The system's functionality is built on a robust backend pipeline that handles both document ingestion and mindmap evaluation.

1. Document Upload Processing:

- A user uploads a document through the UI.
- The file is stored on the backend, and a background process is triggered. This ensures the UI remains responsive while document analysis occurs asynchronously.

2. Document Processing Pipeline:

- Chunking: The document is split into smaller, manageable chunks (e.g., 500-character segments with 10% overlap).
- AI Topic Extraction: Each chunk is passed to the AI model, which extracts related topics and short descriptions/details.
- Topic Aggregation: Topics from all chunks are merged, and duplicate topics are identified and removed.
- Database Storage**:** The final topics and descriptions are stored in the database. Additionally, vector embeddings of each chunk are stored to support RAG, enabling semantic retrieval during evaluation.

3. Mind Map Evaluation Workflow:

- Step 1: Fetch Topics: When a mind map JSON is submitted, the backend retrieves all stored "ground-truth" topics from the database based on the associated Module and Exercise IDs.
- Step 2: Topic Comparison: The system compares topics from the database with topics from the student's mind map JSON to produce lists of matched and unmatched topics.
- Step 3: RAG Retrieval: For matched topics, RAG is used to retrieve additional context from the stored embeddings to enrich the evaluation.

- Step 4: AI Evaluation: The AI receives the matched topics (with details and RAG context) and unmatched topics. It then evaluates the submission, returning a point score and detailed reasoning.
- Step 5: Pass/Fail Decision: A configurable threshold is applied to the score to mark the submission as a "Pass" or "Fail."
- Step 6: Response to UI: The backend returns a JSON response containing the evaluation points, AI reasoning, and Pass/Fail status.

4. Advanced AI Capability: Model Fine-Tuning

A central pillar of AgenticContentFlow's performance is its specialized model fine-tuning process, which was undertaken to overcome the limitations of using a base model.

Initial Challenges with Base Models

The initial system attempted to use a base LLaMA 3B model for evaluation but faced critical issues:

- The model failed to correctly apply the defined scoring formula.
- It would suggest invalid or illogical reasoning for topic connections.

This highlighted the need for a specialized model.

Dataset Creation

1. **Data Generation:** We generated 20,000 artificial samples of mind map evaluation data to ensure high coverage of diverse educational scenarios, as reliable, real-world datasets for mind maps were not available. The dataset included multiple variations, from 10% to 100% semantic similarity within a specific domain.

2. **Data Format:** Each data sample was structured to include ‘instruction’, ‘input’ (with context, flow topics, edges, and matching criteria), and ‘output’ (with feedback and points), forming a complete training example.

Fine-Tuning Process

- **Model Selection:** The team selected ‘unsloth/Phi-3-mini-4k-instruct-bnb-4bit’ as the base model due to its good balance of capability and speed, a 4K context window for handling full prompts, and memory efficiency via 4-bit quantization.

- **Training Data:** A subset of 5,000 high-quality samples was selected from the larger 20,000 dataset for the fine-tuning phase.
- **Configuration:** The Unsloth library was used for optimized training with the following parameters:
    - LoRA Parameters: Rank ('r=64'), alpha ('lora_alpha=128'), and target modules for efficient fine-tuning.
    - Training Arguments: Batch size of 2, learning rate of 2e-4, 3 epochs, and a max sequence length of 3084 tokens.
    - Training: The model was trained on a single GPU, resulting in ~119.5 million trainable parameters (3.03% of the total model), which is efficient for specialization.
    - Model Artifact: The resulting fine-tuned model, named 'kiran2.0', was saved in the 'safetensor' and 'GGUF' formats for efficient local inference.

5. Development Journey: Experimentation & Iteration

Evidence of Experimentation

The development process was highly iterative:

- **Model Selection:** Benchmarked Llama 3.2 against Phi-3. While Llama demonstrated stronger creativity and reasoning capabilities, Phi-3 proved easier to fine-tune into *kiran2.0* for structured feedback tasks.
  We also evaluated fine-tuning larger variants such as Llama 8B and Llama 7B due to their strong reasoning performance. However, these experiments could not be fully pursued because available Google Colab VRAM was insufficient, leading to memory constraints during training.
- **Logic Refinement:** Early versions used strict string matching, which failed on synonyms. This was improved by moving to Sentence-Transformers for semantic matching.
- **Scoring Adjustment:** The penalty for "missing sub-details" was reduced to ensure the feedback focused more on high-level conceptual mastery.

Key Challenges & Solutions

- **Tensor Shape Errors:** Encountered Llama 3.2 runtime errors during long-context evaluation.

  Solution: Implemented a windowed context approach to handle longer conversations by keeping only the most relevant recent information. Also adjusted the response settings in Ollama (like temperature and token limits) to make the model's answers more consistent and better suited for structured feedback.
  Additionally, instead of sending the full document every time, a topic-matching system is used, and fetch only the mentioned topics information using Retrieval-Augmented

Generation. This helps improve accuracy while reducing the token consumption for large data sending to the model.

- **Topic Ambiguity and Duplication:** The system initially failed to separate or correctly match semantically similar topics (e.g., "Artificial Intelligence" vs "AI", "ML" vs "Machine Learning"). This led to duplicate topic entries in the database and inaccurate matching during evaluation, where the system would treat variations of the same concept as distinct or, conversely, incorrectly merge unrelated topics.

  Solution: Implemented a semantic deduplication layer using Sentence-Transformers with a cosine similarity threshold (default 0.85) to identify and merge near-identical topics during document processing. For evaluation matching, the system now normalizes topic names (lowercase, whitespace stripping) and applies semantic similarity scoring rather than exact string matching, allowing "AI" and "Artificial Intelligence" to be correctly recognized as the same core concept.

- **Base Model Evaluation Inaccuracy:** The base LLaMA 3B model failed to correctly apply scoring rules and provided invalid reasoning.

  Solution: This led to the creation of the specialized 'kiran2.0' model through the fine-tuning process described in Section 4.

- **Google Colab GPU Timeout:** During the fine-tuning process, Google Colab sessions would frequently time out due to GPU usage limits, causing the training process to restart from the beginning. This resulted in significant wasted time and computational resources, as the model could not be trained to completion within a single session.

  Solution: Implemented a checkpoint saving mechanism using Google Drive integration. This saves model checkpoints after each 50 steps and removes the previously saved checkpoint automatically to save the Google Drive space.
  By mounting Google Drive and configuring the trainer to resume from the latest saved checkpoint, the fine-tuning process continues seamlessly across multiple Colab sessions without losing progress.

- **Dual Format Model Compatibility:** The fine-tuning pipeline produced models in the SafeTensors format, which offered optimal performance and integration within the Google Colab training environment. However, the local inference engine (Ollama) required models in the GGUF format for efficient CPU/GPU execution on local machines. This incompatibility created a gap between the training and deployment environments.

  Solution: Automated the generation of both model formats as part of the post-training pipeline. After fine-tuning with Unsloth, the model was exported and saved in SafeTensors format for Colab-based testing and validation. A secondary conversion step was added for producing a GGUF quantized version specifically for local deployment with Ollama. This

dual-format approach ensured seamless compatibility across both the Colab and Local environments.

6. Comparison between Llama3.2 and Kiran2.0

To quantitatively evaluate the effectiveness of our fine-tuning process, we conducted a comparative analysis between the base model (Llama 3.2) and our specialized fine-tuned model (Kiran 2.0). A test set of 100 mind map evaluation samples was used, with a correctness-threshold of ±0.15 points from the ground truth.

Summary Statistics

Metric	Llama 3.2 (Base)	Kiran 2.0 (Fine-Tuned)
Correct Evaluations	42 / 100 (42.0%)	60 / 100 (60.0%)
Outperformance	Baseline	Outperformed Llama 3.2 in 55/100 samples (55.0%)

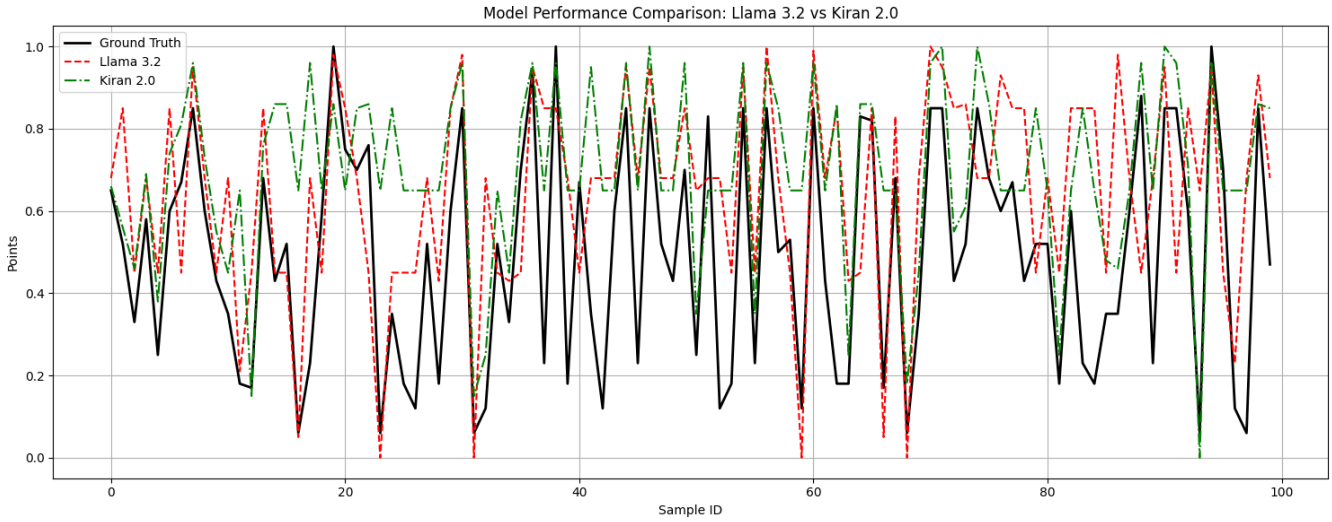
Performance Analysis

The results clearly demonstrate the success of the fine-tuning process. Kiran 2.0 achieved a significantly higher correctness rate (60%) compared to the base Llama 3.2 model (42%).

- **Correctness:** Kiran 2.0 was correct in 60% of the evaluations, an 18-percentage point improvement over the base model. This indicates a much better understanding and application of the defined scoring rubric.
- **Consistency:** The fine-tuned model showed a stronger tendency to produce scores closer to the ground truth. While Llama 3.2 often produced scores at the extremes (either too high or too low), Kiran 2.0's predictions were more tightly clustered around the expected values.
- **Reasoning Quality:** Llama 3.2 frequently generated generic or illogical reasoning, often referencing topics outside the domain (e.g., "Deep Learning -> Computer Vision" in a "Biochemistry" context). In contrast, Kiran 2.0's feedback was more focused, contextually relevant, and aligned with the specific course material, demonstrating a superior understanding of the evaluation task.

- **Response Time Efficiency:** Kiran 2.0 demonstrated significantly faster processing, completing evaluation of 100 samples in 2 hr. 18 min, compared to 7 hr. 10 min for Llama 3.2. This improvement highlights better efficiency and makes Kiran 2.0 more suitable for real-time or large-scale evaluation scenarios.

Metric	Llama 3.2 (Base)	Kiran 2.0 (Fine-Tuned)
Correct Evaluations	42 / 100 (42.0%)	60 / 100 (60.0%)
Score Distribution	Prone to overestimation and underestimation.	Closer to ground actual scores.
Reasoning Quality	Often generic, illogical, or out-of-domain.	Focused, contextually relevant, and aligned.
Response Time	7hr 10min for 100 data	2hr 18min for100 data



This quantitative analysis validates that the fine-tuning process successfully specialized the model for the educational concept mapping task, making Kiran 2.0 the superior choice for deployment.

7. Lessons Learned & Future Work

Key Takeaways

- **Choosing the *right* model matters more than the most powerful one:** While Llama (8B and 7B) showed better creativity and reasoning, it was harder to control for structured evaluation tasks. Phi-3 proved more practical for fine-tuning and consistent outputs.
- **RAG is essential for scalable and accurate systems:** Sending full documents was inefficient and costly. Switching to topic-based retrieval (RAG) improved both accuracy and performance.
- **Context management is critical for LLM stability:** Long input causes errors and inconsistent outputs. Windowed context and parameter tuning in Ollama improved reliability.
- **Creating Synthetic data:** Created 20K artificial samples due to lack of real datasets.
- **Hybrid systems are better:** Combine deterministic rule-based scoring with flexible LLM reasoning for hybrid evaluation.
- **Semantic understanding is crucial:** Exact matching fails with language variability; use embeddings + cosine similarity for semantic understanding.
- **Pipeline design and async processing improve UX:** Keep UIs responsive with async processing, decoupled architectures are important for production AI.
- **Iteration is mandatory:** AI systems demand continuous iteration, model selection, logic, scoring, and fine-tuning all evolve through cycles, not one-time builds.

Future Work

1. **Improve Model Capability & Efficiency:** Fine-tune larger or more advanced models (e.g., models with 8B, 14B or 20B perameters).
2. **Smarter topic matching:** Move from basic topic matching to semantic search with re-ranking.
3. **Graph-Aware AI Evaluation:** Move beyond text to GNNs/structure-aware embeddings to encode hierarchy via graph depth, directionality through edge orientation, and relationship quality via edge attributes or attention mechanisms.
4. **Better Dataset & Continuous Learning:** Replace synthetic data with real student submissions, add active learning so the model iteratively improves from user corrections, improving robustness and adaptation.

8. Conclusion

Mindmap is not just a tool for students; it is a scalable platform for visual learning. By combining specialized fine-tuned models (kiran2.0') with robust RAG infrastructure, it will provide a better

educational experience that is cost-effective, and pedagogically sound. The project successfully transitioned from a concept to a functional system with a fully implemented backend, a responsive frontend, and an AI evaluation core that was significantly improved through a dedicated fine-tuning process. This platform lays a strong foundation for future enhancements that can further enrich the learning experience.